SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI-602105

NAME	AKSHAYA R S
COURSE CODE	CSA0209
REG NO	1924260209
GITHUB LINK	https://github.com/akshaya12258-droid/CSA0209-C-PROGRAMMING-1924260209

ADVANCED PROBLEM SOLVING AND ALGORITHM DESIGN

Q1 – Searching a Library Catalogue (6 Marks)

1. Problem Understanding

The catalogue stores accession numbers. Linear search can inspect an unsorted array, while binary search can search an ascending sorted array by repeatedly discarding half of the remaining range. The trial contains 10 books; the full catalogue may contain 100,000 books. A comparison means one test of an array element against the requested key.

2. Aim and Objectives

• Implement linear and binary search and count key comparisons.

• Compare successful searches at the beginning, middle and end with an unsuccessful search.

• Explain comparison counts mathematically and assess whether keeping the catalogue sorted is worthwhile.

3. Algorithm Explanation

Linear search: examine elements from index 0 onward. Stop when the key is found or the end is reached. It works whether or not the array is sorted.

Binary search: maintain low and high indices. Inspect the middle element. If it is smaller than the key, continue in the right half; if larger, continue in the left half. It requires ascending sorted data because these comparisons must reliably tell which half can contain the key.

The expression `low + (high - low) / 2` is preferred to `(low + high) / 2` because low + high can overflow an integer for very large indices, whereas high - low remains within the active range.

4. Pseudocode

```
LINEAR_SEARCH(A, n, key)
comparisons = 0     for i = 0 to n -
1         comparisons = comparisons +
1         if A[i] == key
return i, comparisons     return -1,
comparisons

BINARY_SEARCH(A, n, key)   // A is sorted ascending
low = 0; high = n - 1; comparisons = 0     while
low <= high
        mid = low + (high - low) / 2

        comparisons = comparisons + 1         if
A[mid] == key: return mid, comparisons
comparisons = comparisons + 1         if A[mid] <
key: low = mid + 1         else: high = mid - 1
return -1, comparisons
```

For a comparison-count convention that counts each three-way decision as one comparison of A[mid] with key, count one comparison per inspected middle element. The completed supplied program uses this convention, matching its expected output. The pseudocode above can be read with the same convention by treating the equality/order decision as one three-way comparison.

5. C Program – Linear and Binary Search

```
#include <stdio.h>

int linear_steps, binary_steps;

int linear_search(const int a[], int n, int key) {
linear_steps = 0;     for (int i = 0; i < n; i++)
```

```
{         linear_steps++;         if (a[i] == key)
return i;     }     return -1; }

int binary_search(const int a[], int n, int key) {
    int low = 0, high = n - 1;
binary_steps = 0;     while (low <= high)
{         int mid = low + (high - low) /
2;         binary_steps++;         if
(a[mid] == key) return mid;         if
(a[mid] < key) low = mid + 1;
else high = mid - 1;     }     return -1;
}

int main(void) {     int unsorted[] = {1190, 1021, 1533, 1103, 1402, 1048, 1305,
1222, 1460, 1377};     int sorted[] = {1021, 1048, 1103, 1190, 1222, 1305, 1377,
1402, 1460, 1533};     int keys[] = {1021, 1305, 1533, 1400};     int n =
sizeof(sorted) / sizeof(sorted[0]);

    for (int k = 0; k < 4; k++) {         int p1 =
linear_search(unsorted, n, keys[k]);         printf("Linear:
%d -> index %d, comparisons = %d\n",                keys[k],
p1, linear_steps);         int p2 = binary_search(sorted, n,
keys[k]);         printf("Binary: %d -> index %d,
comparisons = %d\n",                keys[k], p2,
binary_steps);     }     return 0; }
```

6. Completed TODO Program and Expected Output

```
#include <stdio.h>
int steps;

int binary_search(const int a[], int n, int key) {     int low = 0,
high = n - 1;     steps = 0;     while (low <= high) {         int
mid = low + (high - low) / 2;         steps++;                 /* one
element-to-key comparison */         if (a[mid] == key) return mid;
if (a[mid] < key) low = mid + 1;         else high = mid - 1;     }
return -1; }

int main(void) {     int accession[] =
{1021,1048,1103,1190,1222,1305,1377,1402,1460,1533};     int n = sizeof
accession / sizeof accession[0];     int keys[] = {1021,1305,1533,1400};
for (int k = 0; k < 4; k++) {
        int pos = binary_search(accession, n, keys[k]);
        printf("Book %d -> index %d, comparisons = %d\n", keys[k], pos, steps);
}     return 0; }
```

Expected output (matches the supplied assignment):

```
Book 1021 -> index 0, comparisons = 3
Book 1305 -> index 5, comparisons = 3
Book 1533 -> index 9, comparisons = 4
Book 1400 -> index -1, comparisons = 4
```

The first midpoint is index 4 (1222). For 1021, the next midpoint is index 1 (1048), then index 0. For 1305, the inspected indices are 4, 7 and 5. For 1533, the inspected indices are 4, 7, 8 and 9. For missing 1400, the search inspects 4, 7, 8 and 9 before the range becomes empty.

7. Mathematical Analysis

Search	Best comparisons	Worst comparisons	Average / typical
Linear	1 (first element)	n	Successful: (n+1)/2 if each position is equally likely; unsuccess n
Binary	1 (middle element)	ceil(log2(n+1))	O(log n); exact average depends on n and the key distribution

For linear search, a successful key at position i (0-based) takes i+1 comparisons. Averaging positions 1 through n gives (1+2+...+n)/n = (n+1)/2. An unsuccessful search examines all n elements.

For binary search, each comparison reduces the remaining candidate range to at most half. Its maximum number of inspected midpoints is ceil(log2(n+1)). For n = 100,000, ceil(log2(100001)) = 17. Linear search may require 100,000 comparisons; its average successful search is 50,000.5 and an unsuccessful search needs 100,000.

A missing key can require as many comparisons as the worst successful search because binary search must keep halving the possible range until it is empty. Some unsuccessful keys fall out earlier, but the final empty-range proof can take the full search depth.

8. Student A – Design and Mathematical Analysis

The pseudocode and formulas above satisfy the design task. Binary search needs sorted data: if the values are arbitrary, a smaller midpoint does not prove that the key must be to its right. Sorting first costs time, so the catalogue should be maintained or periodically rebuilt in sorted order. The overflow-safe midpoint formula is preferred for robust index arithmetic.

9. Student B – Experimental Investigation

Generate ascending arrays of sizes 1,000, 10,000 and 100,000. For each size, search for (a) the first, middle and last existing values and (b) a value outside the range. For linear search, place keys at the same positions in the array; for binary search use the sorted array. Reset the comparison counter before each search. Record the exact counter, not elapsed time alone.

n	Linear successful (first / middle / last)	Linear missing	Binary maximum (successful or missing upper bound)
1,000	/ 500 / 1,000	1,000	10
10,000	/ 5,000 / 10,000	10,000	14
100,000	/ 50,000 / 100,000	100,000	17

These are predicted comparison counts from the algorithms, not measured wall-clock results. Plot n on the horizontal axis and comparisons on the vertical axis. The linear-search worst-case line rises proportionally to n; the binary-search line rises very slowly with log2(n). To report actual experimental measurements, run the program on the lab computer and replace or supplement the predictions with the recorded counter values. The expected result agrees with Student A's mathematical prediction.

10. Shared Group Work

The group implements both functions, uses an unsorted copy for linear search and an ascending copy for binary search, tests the four required positions and missing key, and records each counter. The binary-search function above is the completed TODO implementation.

11. Joint Conclusion / Engineering Decision

Keeping the catalogue sorted is usually worthwhile for a large, frequently searched library, but the decision depends on update frequency and search frequency. Required information includes: books added or removed per day; number of searches per day; cost and time to insert while preserving order; whether additions can be batched; memory constraints; and whether a database or index is already available. If searches greatly outnumber updates, sorted storage is attractive. If updates are very frequent and searches are rare, a hash table or database index may be better.

12. Alternative Concept

Hash table with linear probing: map accession number to a slot using key mod table_size. If occupied, inspect subsequent slots until the key or an empty slot is found. Average search is O(1) when the load factor is controlled and the hash function distributes keys well; worst case is O(n) due to clustering. Extra space is O(table size), typically O(n).

Interpolation search: estimate the likely position from the key's numeric value and the endpoint values. On uniformly distributed sorted numeric keys, average time can be O(log log n); worst case is O(n), for example with highly uneven distributions. It uses O(1) auxiliary space. Binary search guarantees O(log n) worst-case time and O(1) auxiliary space.

13. Advantages, Disadvantages and Limitations

• Linear search is simple and works on unsorted data, but scales poorly for large catalogues.

- Binary search is fast and uses constant auxiliary space, but the data must remain sorted.
- A hash table is fast on average but needs extra storage and careful collision/load-factor management.
- Interpolation search can be faster on suitable uniform numeric data, but its worst case is weaker.

Q2 – Sorting the Examination Hall Seating Register (7 Marks)

1. Problem Understanding and Aim

The register must be sorted by register number. Input may be sorted, nearly sorted, random or reverse sorted. Three elementary comparison sorts are required: bubble sort with early exit, selection sort and insertion sort. The counters should record key comparisons and swaps or shifts.

2. Algorithm Explanation

Bubble sort: compare adjacent values and swap if out of order. After each pass, the largest remaining value reaches the end. An early-exit flag records whether any swap occurred; if no swap occurred, the array is already sorted and the algorithm stops.

Selection sort: find the smallest value in the unsorted suffix and swap it into the next output position. It performs the same number of key comparisons regardless of input order.

Insertion sort: take each next value and shift larger values right until the correct gap is found. It is efficient for nearly sorted data because only a few elements need to move.

3. Pseudocode

```
BUBBLE(A, n)     for end = n-1 down to 1         swapped =
false         for j = 0 to end-1             comparisons++
if A[j] > A[j+1]: swap; swaps++; swapped = true         if
not swapped: break

SELECTION(A, n)     for i = 0 to n-2
min = i         for j = i+1 to n-1
comparisons++             if A[j] < A[min]: min
= j         if min != i: swap A[i], A[min];
swaps++

INSERTION(A, n)     for i =
1 to n-1         key =
A[i]; j = i-1         while
j >= 0
comparisons++
            if A[j] > key: A[j+1] = A[j]; shifts++; j-
else: break         A[j+1] = key
```

4. Complete C Program

```
#include <stdio.h>

void bubble_sort(int a[], int n, long *cmp, long *swaps) {
*cmp = *swaps = 0;
    for (int end = n - 1; end > 0; end--) {
int swapped = 0;
        for (int j = 0; j < end; j++) {
(*cmp)++;             if (a[j] > a[j +
1]) {
                int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
(*swaps)++; swapped = 1;
            }
}
        if (!swapped) break;
} }
void selection_sort(int a[], int n, long *cmp, long *swaps) {
*cmp = *swaps = 0;     for (int i = 0; i < n - 1; i++) {
int min = i;         for (int j = i + 1; j < n; j++) {
(*cmp)++;             if (a[j] < a[min]) min = j;
        }         if (min != i) {             int t
= a[i]; a[i] = a[min]; a[min] = t;
(*swaps)++;
        }
    } } void insertion_sort(int a[], int n, long *cmp, long
*shifts) {     *cmp = *shifts = 0;     for (int i = 1; i < n;
i++) {         int key = a[i], j = i - 1;         while (j >=
0) {             (*cmp)++;             if (a[j] > key) {
a[j + 1] = a[j]; (*shifts)++; j--;             } else break;
}         a[j + 1] = key;     } } void print_array(const int
a[], int n) {     for (int i = 0; i < n; i++) printf("%d ",
a[i]);     printf("\n"); } int main(void) {     int x[] = {42,
17, 31, 8, 25, 13};     int y[] = {42, 17, 31, 8, 25, 13};
int z[] = {42, 17, 31, 8, 25, 13};     long c, m;
```

```
bubble_sort(x, 6, &c, &m);     printf("Bubble: ");
print_array(x, 6);     printf("Comparisons=%ld swaps=%ld\n",
c, m);     selection_sort(y, 6, &c, &m);
printf("Selection: "); print_array(y, 6);
printf("Comparisons=%ld swaps=%ld\n", c, m);
insertion_sort(z, 6, &c, &m);     printf("Insertion: ");
print_array(z, 6);     printf("Comparisons=%ld shifts=%ld\n",
c, m);     return 0; }
```

5. Sample Output and Hand Trace

```
Bubble: 8 13 17 25 31 42
Selection: 8 13 17 25 31 42
Insertion: 8 13 17 25 31 42
```

The exact counter values depend on the chosen counter convention; the code above counts each value comparison and counts an insertion shift only when an existing item moves right. On the six-value input, bubble sort performs passes until no swaps remain; selection sort selects 8, then 13, then 17, then 25, then 31; insertion sort grows a sorted prefix one value at a time.

Input order	Bubble sort behaviour	Selection sort behaviour	Insertion sort behaviour
Sorted	n−1 comparisons, 0 swaps due to	arly exitn(n−1)/2 comparisons; 0 waps	n−1 comparisons, 0 shifts
Nearly sorted	few swaps; may still scan long pas	esfixed n(n−1)/2 comparisons	near-linear; shifts relate to inversions
Random	quadratic comparisons; swaps = in	ersionsquadratic comparisons; a most n−1	swquadratic average; shifts = inversionsps
Reverse	quadratic comparisons and n(n−1)/	2 swapsquadratic comparisons; a most n−1	swquadratic comparisons and n(nps −1)

/2 shifts

6. Stability

Bubble sort (when it swaps only if left > right) is stable: equal values are not swapped, so their relative order remains. Insertion sort (when it shifts only values greater than the key) is stable for the same reason. Standard selection sort is not stable because swapping a distant minimum into place can change the order of equal records. Stability matters when records have equal marks: a stable sort can preserve a previous order such as registration time or roll number.

7. Student A – Algorithm Design and Tracing

The pseudocode above is the design. The early-exit flag avoids unnecessary later passes when a complete pass makes no swaps. Insertion sort is fast on nearly sorted data because its work is proportional to the number of inversions plus the basic pass overhead. If only a few late registrations are out of order, only a small number of shifts may be needed.

8. Student B – Comparative Measurement

Run the program after generating four input arrangements at n = 100, 1,000 and 10,000. Keep the same input copy for each algorithm. Record comparisons and swaps/shifts separately. The following table gives theoretical comparison counts for sorted and reverse-sorted inputs; random and nearly-sorted counts depend on the generated permutation.

n	Sorted bubble	Reverse bubble	Selection (all orders)	Sorted insertion	Reverse insertion
100	99	4,950	4,950	99	4,950
1,000	999	499,500	499,500	999	499,500
10,000	9,999	49,995,000	49,995,000	9,999	49,995,000

For random distinct values, expected inversions are n(n−1)/4; bubble swaps and insertion shifts are therefore approximately that amount on average. For a nearly sorted test, deliberately move 20 selected late registrations from the end into earlier positions and record the resulting inversion count. Do not call a wall-clock value a measured result unless the program was actually timed. For graphs, make one plot per metric: x-axis = n; y-axis =

comparisons or swaps/shifts; use separate series for the three algorithms and separate panels or charts for each input order.

If an observation differs from expectation, check whether the input was truly sorted/reverse sorted, whether counters were reset, whether swaps or shifts were counted consistently, and whether the compiler optimised the timing loop.
Compare operation counts before comparing elapsed time.

9. Shared Group Work

Use identical input arrangements and make a fresh copy before each sort. Verify ascending order after sorting. Record the comparison and movement counters, and plot the measurements. Include the generated input seed so random experiments can be repeated.

10. Joint Conclusion / Engineering Decision

For a register that is already sorted except for 20 late registrations, insertion sort is the best of these three simple methods. It preserves the existing order and shifts only the values that must move. Bubble sort with early exit can also be reasonable, but it may make several full passes. Selection sort performs quadratic comparisons regardless of how nearly sorted the input is.

11. Alternative Concept – Counting Sort

If register numbers are integers in a known compact range of size k, create count[k] initialised to zero. For each register value x, increment count[x−minimum]. Then output each value the recorded number of times. Time is O(n+k), extra space O(k), and it is not a comparison sort. It is useful when k is not much larger than n. It is unsuitable when the range is huge or sparse, when values are not discrete integers, or when the original records must be retained without additional stable-record handling. Stable counting sort can preserve equal-key record order by using cumulative counts and an output array.

12. Complexity Analysis

Algorithm	Best time	Average time	Worst time	Auxiliary space	Stable?
Bubble (early exit)	O(n)	O(n²)	O(n²)	O(1)	Yes
Selection	O(n²)	O(n²)	O(n²)	O(1)	No
Insertion	O(n)	O(n²)	O(n²)	O(1)	Yes
Counting	O(n+k)	O(n+k)	O(n+k)	O(k) or O(n+k) for stable	outputCan be

(standard swap form)

Q3 – Merging Two Merit Lists (7 Marks)

1. Problem Understanding

Campus A and Campus B each have a merit list sorted from highest score to lowest. The goal is to combine them into one descending list without sorting all scores again. Lists may have different lengths, equal scores or one empty list.

2. Aim and Algorithm

Maintain one index for each input. Compare the two current scores and copy the larger score to the output. Advance only the index whose value was copied. When either input is exhausted, copy the remaining tail of the other input.

3. Pseudocode

```
MERGE(A, m, B, n, OUT)      i = 0; j = 0;
k = 0      while i < m and j < n
if A[i] >= B[j]              OUT[k] =
A[i]; i++          else
OUT[k] = B[j]; j++          k++      while
i < m: OUT[k] = A[i]; i++; k++      while
j < n: OUT[k] = B[j]; j++; k++
return k
```

4. Completed C Program

```
#include <stdio.h>

int merge(const int a[], int na, const int b[], int nb, int out[]) {
int i = 0, j = 0, k = 0;      while (i < na && j < nb) {          if
(a[i] >= b[j]) out[k++] = a[i++];          else out[k++] = b[j++];
}      while (i < na) out[k++] = a[i++];      while (j < nb) out[k++] =
b[j++];      return k; } int main(void) {      int campusA[] = {98, 91,
85, 72};      int campusB[] = {95, 91, 80};      int merged[7];      int
n = merge(campusA, 4, campusB, 3, merged);      for (int i = 0; i < n;
i++) printf("%d ", merged[i]);      printf("\n");      return 0; }
```

Expected output:

```
98 95 91 91 85 80 72
```

5. Dry Run and Verification

Step	A current	B current	Copied	Output so far
1	98	95	98	98
2	91	95	95	98 95
3	91	91	91 from A	98 95 91
4	85	91	91 from B	98 95 91 91
5	85	80	85	98 95 91 91 85
6	72	80	80	98 95 91 91 85 80

7	72	—	72 (remaining A)	98 95 91 91 85 80 72

With an empty A list, the main while condition is false immediately and the second tail-copy loop copies all of B. The symmetric case works the same way. With all scores equal, the >= choice copies from A first and then B; the merged scores remain correctly sorted.

6. Student A – Analysis and Comparison

Every iteration of the main loop copies one element and advances one pointer. Therefore, each element is copied once. A comparison occurs only while both lists still have elements.

For non-empty lists of sizes m and n, the maximum comparisons are m+n−1: this occurs when the lists remain interleaved until one is exhausted. The minimum is min(m,n), when all elements of one list are selected before the other list is exhausted. If either list is empty, comparisons are zero. The number of output writes is exactly m+n.

m = n	Merge maximum comparisons	Join then comparison-sort (rough n log■n scale, N=2m)
1,000	1,999	about 21,932 comparisons at N log■N
10,000	19,999	about 285,754 comparisons at N log■N

The qsort comparison count is implementation- and input-dependent, so N log2 N is a scale estimate rather than an exact qsort count. Merge is O(m+n) and uses O(m+n) output storage. Sorting the concatenated N=m+n elements is O(N log N) and generally requires extra stack or implementation-dependent workspace.

7. Student B – Merge Design and Correctness

At each main-loop step, the larger current score is the next possible largest unused score, because each input is already descending. Thus copying it cannot place a smaller value before a larger remaining value. Each step advances one input index, so no element is compared repeatedly after it is consumed. Once one list ends, the other list is already sorted; copying its remaining suffix preserves order.

8. Shared Group Work

The group tests different list lengths, equal scores, descending-order verification and empty-list cases. The output array must have capacity at least m+n. Validate that each input is already sorted in descending order before calling merge.

9. Joint Engineering Decision

An in-place merge may be preferred when allocating a third array of m+n scores is impossible or memory is extremely constrained. It is more complex: shifting elements to make room can require O(m·n) time for a simple method. A more advanced in-place merge can improve this bound but is harder to implement and verify. The third-array approach is simple, predictable and linear.

10. Alternative Concept and Complexity

Concatenate and qsort: copy both lists into one array and call qsort with a descending integer comparator. Time O((m+n) log(m+n)); extra memory depends on qsort implementation and recursion/stack requirements, in addition to the concatenated array.

Simple in-place merge: insert elements from the second list into the first by shifting larger runs. A straightforward method can take O(m·n) time and O(1) extra space. It avoids the output array but may perform many element moves.

Method	Time	Extra space beyond inputs/output
Third-array merge	O(m+n)	O(m+n)
Concatenate + qsort	O((m+n) log(m+n))	O(m+n) for concatenation, plus qsort workspace
Simple in-place insertion merge	O(m·n)	O(1)

Q4 – Two-Dimensional Arrays: District Rainfall Records (7 Marks)

1. Problem Understanding and Objectives

The rainfall table contains 38 districts and 12 months. Required results are annual total per district, average rainfall for each month across the state, the wettest district-month and districts whose annual totals are below 60% of the state annual average per district. The phrase “state average” should be defined clearly; this solution uses the average annual rainfall per district, equal to total rainfall across all district-month cells divided by 38.

2. Array Design and Memory Layout

In C, a built-in two-dimensional array is stored in row-major order. All 12 months of district 0 are contiguous, followed by all 12 months of district 1, and so on. For an array declared `int rain[38][12]`, the address of element rain[i][j] is:

Address(rain[i][j]) = base + ((i × 12) + j) × sizeof(int)

The row-wise loop has the inner loop over months j, matching contiguous memory. A column-wise traversal jumps 12 integers between adjacent accesses and can use CPU cache less effectively. For large arrays, cache locality can make row-wise traversal faster even though both visit the same number of elements.

3. Pseudocode

```
FOR each district i:
    total[i] = sum rain[i][j] for j = 0..11 FOR each month j:
monthly_average[j] = sum rain[i][j] for i = 0..37 / 38 maximum
= largest rain[i][j], retaining its district and month
state_average_per_district = sum of all cells / 38 FOR each
district i:
    if total[i] < 0.60 * state_average_per_district: report i
```

4. Complete C Program with Separate Functions

```
#include <stdio.h>

#define DISTRICTS 38
#define MONTHS 12

void district_totals(const int rain[DISTRICTS][MONTHS], long totals[]) {
for (int i = 0; i < DISTRICTS; i++) {         totals[i] = 0;
        for (int j = 0; j < MONTHS; j++) totals[i] += rain[i][j];     } }
void monthly_averages(const int rain[DISTRICTS][MONTHS], double avg[]) {
for (int j = 0; j < MONTHS; j++) {         long sum = 0;         for (int
i = 0; i < DISTRICTS; i++) sum += rain[i][j];         avg[j] =
(double)sum / DISTRICTS;     } }
void maximum_rain(const int rain[DISTRICTS][MONTHS],
int *max_value, int *district, int *month) {     *max_value =
rain[0][0]; *district = 0; *month = 0;     for (int i = 0; i <
DISTRICTS; i++)         for (int j = 0; j < MONTHS; j++)
if (rain[i][j] > *max_value) {                 *max_value =
rain[i][j]; *district = i; *month = j;
            } } void deficient_districts(const long totals[],
double threshold) {     printf("Deficient districts (0-based
index): ");     for (int i = 0; i < DISTRICTS; i++)         if
(totals[i] < threshold) printf("%d ", i);     printf("\n"); } int
main(void) {
    int rain[DISTRICTS][MONTHS];     long
totals[DISTRICTS], grand_total = 0;
double avg[MONTHS];     int maxv, maxd,
maxm;
    /* Read 38 rows, each containing 12 non-negative monthly values. */
for (int i = 0; i < DISTRICTS; i++)         for (int j = 0; j < MONTHS;
j++)             if (scanf("%d", &rain[i][j]) != 1 || rain[i][j] < 0) {
printf("Invalid rainfall input\n"); return 1;             }

    district_totals(rain, totals);     for (int i = 0; i <
DISTRICTS; i++) grand_total += totals[i];
monthly_averages(rain, avg);     maximum_rain(rain, &maxv, &maxd,
&maxm);

    for (int i = 0; i < DISTRICTS; i++)         printf("District %d annual
total = %ld\n", i + 1, totals[i]);     for (int j = 0; j < MONTHS; j++)
printf("Month %d average = %.2f\n", j + 1, avg[j]);     printf("Maximum =
%d mm at district %d, month %d\n",             maxv, maxd + 1, maxm + 1);
deficient_districts(totals, 0.60 * ((double)grand_total / DISTRICTS));
return 0; }
```

5. Hand Verification on a 3 × 4 Array

District	Month 1	Month 2	Month 3	Month 4	Row total
A	10	20	30	40	100
B	20	30	40	50	140
C	30	40	50	60	180
Monthly average	20	30	40	50	—

The grand total is 420; the average annual total per district is 420/3 = 140. Sixty percent is 84, so none of the three districts is deficient. The maximum cell is 60 at District C, Month 4. This hand-check validates the same formulas on a smaller shape.

6. Student A – 3-D Extension

A multi-year dataset can be declared `int rain[YEARS][38][12]`. The address offset for rain[y][i][j] is ((y × 38 × 12) + (i × 12) + j) elements from the base. The dimensions represent year, district and month. A row-wise month loop remains the contiguous innermost traversal.

7. Student B – Traversal-Time Experiment

Allocate a 2,000 × 2,000 integer array (about 16 MB when int is 4 bytes). Initialise every element. Time a full row-wise traversal with j as the inner loop, then a column-wise traversal with i as the inner loop. Use the same operation in both loops, repeat several times, and report the median. Compile with the same optimisation setting. The row-wise traversal is expected to be faster because consecutive accesses share cache lines; actual timings must be measured on the test computer.

```
/* Core timing experiment: compile with gcc -O2 traverse.c -o traverse */
#include <stdio.h>
#include <time.h> #define N 2000 static int a[N][N]; int
main(void) {     volatile long long sum = 0;     clock_t
start, end;     for (int i=0;i<N;i++) for (int j=0;j<N;j++)
a[i][j]=i+j;

    start = clock();     for (int i=0;i<N;i++) for (int
j=0;j<N;j++) sum += a[i][j];     end = clock();
printf("Row-wise: %.6f seconds\n",            (double)(end-
start)/CLOCKS_PER_SEC);

    start = clock();     for (int j=0;j<N;j++) for (int
i=0;i<N;i++) sum += a[i][j];     end = clock();
    printf("Column-wise: %.6f seconds\n",
(double)(end-start)/CLOCKS_PER_SEC);
printf("Checksum: %lld\n", sum);     return 0;
}
```

8. Shared Group Work and Joint Conclusion

A 2-D array is clearer and safer than 38 separately named one-dimensional arrays: one loop can process every district, the dimensions are explicit, and the compiler knows the row stride. It also gives contiguous row storage and simple address arithmetic.

9. Alternative Concepts and Complexity

A flat one-dimensional array can store the same cells at index i×12+j. Access is O(1), but the programmer must calculate the index correctly. A pointer-to-row design such as 38 pointers, each allocated to 12 integers, also gives O(1) indexing but requires 38 pointer entries plus separate row allocations and may scatter rows in memory.

Representation	Access time	Extra storage	Memory behaviour
int rain[38][12]	O(1)	No pointer table	Contiguous rows; strong locality
int flat[38*12]	O(1)	No pointer table	Contiguous; manual index calculation
int *rows[38]	O(1)	38 pointers plus allocation metada	ta/paddingRows may be non-contiguous; extr indirection

All required computations take O(38×12) = O(D×M) time. The totals and averages require O(38+12) auxiliary storage in this implementation. Row-major and flat layouts are likely to have similar locality when traversed sequentially; pointer rows can be slower if scattered.

Q5 – Processing Text Without the Library (6 Marks)

1. Problem Understanding

The SMS analyser counts words, converts text to uppercase, reverses a string and checks whether a short code is a palindrome. The first implementation uses character loops without string.h. A second implementation uses standard library functions. Input must be bounded and safely read.

2. Character-Level Design

A C string is a character array ending with the null character `\0`. This terminator marks where the text ends; it is not included in the visible text length. Word counting treats any run of spaces as a separator. This question's supplied TODO counts words separated by spaces; the safer extended implementation below treats spaces and tabs as whitespace.

Do not recompute strlen(s) in every loop condition. strlen scans the string each time, so repeatedly calling it inside a loop can make an otherwise linear loop O(n²). Compute the length once, or scan until '\0'.

`char s[] = "OK"` creates a modifiable array containing O, K and '\0'. `char *s = "OK"` points to a string literal; modifying the literal through the pointer is undefined behaviour, so treat it as read-only (prefer `const char *s`).

3. Pseudocode

```
WORD_COUNT(s)     count = 0; inside = false     for
each character until '\0'         if character is
whitespace: inside = false         else if inside is
false: count++; inside = true     return count

PALINDROME(s)     left = 0; right = length(s)-1     while
left < right         move left past non-letters
move right past non-letters         compare
lowercase(s[left]) and lowercase(s[right])         if
unequal: return false         left++; right-    return
true
```

4. Completed TODO Program (without string.h)

```
#include <stdio.h>
#include <ctype.h>

int word_count(const char s[]) {     int
count = 0, inside = 0;     for (int i =
0; s[i] != '\0'; i++) {
        if (s[i] == ' ' || s[i] == '\t' || s[i] == '\n' ||
s[i] == '\r' || s[i] == '\v' || s[i] == '\f') {
inside = 0;         } else if (!inside) {
count++;             inside = 1;         }     }     return
count; } int is_palindrome(const char s[]) {     int left =
0, right = 0;     while (s[right] != '\0') right++;
right--;
    while (left < right) {         while (left < right &&
!isalpha((unsigned char)s[left])) left++;         while (left < right &&
!isalpha((unsigned char)s[right])) right--;         if (tolower((unsigned
char)s[left]) !=             tolower((unsigned char)s[right])) return 0;
left++; right--;     }     return 1; } void uppercase_in_place(char s[]) {
for (int i = 0; s[i] != '\0'; i++)         s[i] = (char)toupper((unsigned
char)s[i]);
} void reverse_in_place(char s[]) {     int left = 0, right = 0;     while
(s[right] != '\0') right++;     if (right == 0) return;     right--;     while
(left < right) {         char t = s[left]; s[left] = s[right]; s[right] = t;
left++; right--;     } } int main(void) {     printf("Words: %d\n", word_count("
Meet at the library at 5 PM "));     printf("%d %d\n", is_palindrome("Madam, I'm
Adam"), is_palindrome("Hello"));     printf("Tab-separated words: %d\n",
word_count("one\ttwo\tthree"));     char msg[] = "meet me";
uppercase_in_place(msg);     printf("Uppercase: %s\n", msg);
reverse_in_place(msg);     printf("Reversed: %s\n", msg);     return 0; }
```

Expected output for the supplied TODO lines:

```
Words: 7
1 0
```

The added tab test prints `Tab-separated words: 3` because the extended function treats a tab as whitespace. The original assignment comment says spaces; this implementation safely generalises separators to common whitespace characters.

5. Library-Function Version and Safe Input

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define SIZE 256

int count_words_lib(const char *s) {     int count = 0, inside = 0;
for (size_t i = 0; i < strlen(s); i++) { /* demonstration only */
if (isspace((unsigned char)s[i])) inside = 0;         else if
(!inside) { count++; inside = 1; }     }     return count; } int
palindrome_lib(const char *s) {     size_t left = 0, right =
strlen(s);     if (right == 0) return 1;     right--;     while (left
< right) {
        while (left < right && !isalpha((unsigned char)s[left])) left++;
while (left < right && !isalpha((unsigned char)s[right])) right--;
if (tolower((unsigned char)s[left]) !=             tolower((unsigned
char)s[right])) return 0;         left++; right--;     }     return 1; }
int main(void) {
    char input[SIZE], copy[SIZE];
printf("Enter feedback: ");
    if (fgets(input, sizeof input, stdin) == NULL) return 1;
input[strcspn(input, "\n")] = '\0';     snprintf(copy,
sizeof copy, "%s", input);     for (size_t i = 0; copy[i] !=
'\0'; i++)         copy[i] = (char)toupper((unsigned
char)copy[i]);     printf("Words: %d\n",
count_words_lib(input));     printf("Uppercase: %s\n",
copy);     printf("Palindrome: %d\n",
palindrome_lib(input));     return 0; }
```

For efficient library code, calculate `size_t len = strlen(s)` once before the loop rather than calling strlen repeatedly. `strcpy(dest, src)` does not know the destination capacity: copying a long message into a small array writes beyond the array boundary, causing undefined behaviour. Use `snprintf(dest, sizeof dest, "%s", src)`, or carefully use `strncpy` and explicitly add a terminator. `fgets` reads at most size−1 characters and null-terminates the buffer.

gets() was removed from the C standard because it has no buffer-size parameter and cannot prevent an input line from overflowing the destination array.

6. Student A – Character-Level Design

The null terminator is essential because it lets functions know where a string ends without separately storing its length. The word-count pseudocode above changes state only when crossing from whitespace into a non-whitespace character, so several spaces between words do not create extra words.

7. Student B – Library Functions and Safety

Run the loop-based and library-based functions on identical strings: an empty string, leading/trailing spaces, repeated spaces, punctuation, mixed case and tab-separated words. Compare returned counts and palindrome results. For a long source copied to a small destination, strcpy can overrun memory; the safe version must know the destination size. Use fgets for input, remove the newline if present, and use snprintf for bounded copying.

8. Shared Work and Joint Conclusion

Own string functions are useful for learning, custom rules or very small embedded environments. For ordinary production programs, standard library functions are usually clearer, tested and less error-prone. Safety still depends on respecting buffer sizes and character conversion rules.

9. Alternative Concept and Complexity

A loop that calls strlen(s) at every iteration can take O(n²), because each strlen scans up to n characters. A single scan, or one strlen call followed by a loop, takes O(n). Iterative reversal takes O(n) time and O(1) auxiliary space. Recursive reversal takes O(n) time but uses O(n) call-stack space; deep recursion can exhaust the stack.

Operation	Time	Auxiliary space
Word count / uppercase	O(n)	O(1)
Iterative reverse	O(n)	O(1)
Palindrome check	O(n)	O(1)
Repeated strlen in loop	O(n²)	O(1)
Recursive reverse	O(n)	O(n) stack

Q6 – A Directory of Names: Sorting and Searching Strings (7 Marks)

1. Problem Understanding

The directory stores up to 500 resident names, each in a 30-character array. In C, a 30-byte row can store at most 29 visible characters plus the terminating '\0'. The directory must be sorted alphabetically, searched by name and checked for duplicates.

2. Design and Algorithm

Read each name using fgets(names[i], sizeof names[i], stdin), then remove the newline if present. Sort rows using strcmp. Binary search is valid only after sorting using the same comparison rule. After sorting, equal names are adjacent, so one pass can report duplicates.

3. Complete C Program – Copying Names

```
#include <stdio.h>
#include <string.h>

#define MAX_NAMES 500
#define NAME_SIZE 30

void remove_newline(char s[]) {     s[strcspn(s, "\n")] = '\0'; } void
sort_names(char names[][NAME_SIZE], int n, long *calls, long *copies) {
*calls = *copies = 0;     for (int i = 1; i < n; i++) {         char
key[NAME_SIZE];         strcpy(key, names[i]); *copies += strlen(key) + 1;
int j = i - 1;         while (j >= 0) {             (*calls)++;
if (strcmp(names[j], key) > 0) {                 strcpy(names[j + 1],
names[j]);                 *copies += strlen(names[j]) + 1;
j--;             } else break;         }         strcpy(names[j + 1], key);
*copies += strlen(key) + 1;     } } int binary_search(char
names[][NAME_SIZE], int n, const char key[],                   long *calls) {
int low = 0, high = n - 1;     *calls = 0;     while (low <= high) {
        int mid = low + (high - low) / 2;
(*calls)++;
        int c = strcmp(names[mid], key);
if (c == 0) return mid;         if (c <
0) low = mid + 1;         else high =
mid - 1;     }     return -1; }
void show_duplicates(char names[][NAME_SIZE], int n) {
for (int i = 1; i < n; i++)         if
(strcmp(names[i], names[i-1]) == 0)
printf("Duplicate: %s\n", names[i]); } int main(void)
{
    char names[MAX_NAMES][NAME_SIZE], key[NAME_SIZE];
int n;
    long cmp, copies, search_cmp;     printf("Number of residents
(0-%d): ", MAX_NAMES);     if (scanf("%d", &n) != 1 || n < 0 || n
> MAX_NAMES) return 1;     getchar(); /* consume newline after
number */     for (int i = 0; i < n; i++) {         if
(fgets(names[i], NAME_SIZE, stdin) == NULL) return 1;
remove_newline(names[i]);     }     sort_names(names, n, &cmp,
&copies);     printf("Sorted directory:\n");
    for (int i = 0; i < n; i++) puts(names[i]);     printf("strcmp calls in sort =
%ld; characters copied = %ld\n", cmp, copies);     show_duplicates(names, n);
printf("Search name: ");     if (fgets(key, sizeof key, stdin) == NULL) return 1;
remove_newline(key);     int pos = binary_search(names, n, key, &search_cmp);
printf("Search result = %d; comparisons = %ld\n", pos, search_cmp);     return 0;
}
```

Input-length note: fgets stores at most 29 characters in a 30-byte row. A name with 29 visible characters fits. A name with 30 visible characters is truncated to 29 characters in this representation; the program should detect a missing newline and consume the rest of that input line, then report that the name was too long rather than silently accepting a truncated duplicate. For a strict implementation, use a larger temporary line buffer and reject names whose length is 30 or more.

4. Student A – Measurement and Improvement

For n = 100, 500 and 5,000, the supplied fixed-size names array must be enlarged to at least 5,000 rows for the last experiment. Generate names with controlled lengths and repeatable data. Count every strcmp call. Count character

copies as the number of bytes copied including '\0'; the code's counters show the approximate copying cost for insertion sort.

To avoid moving full names, create an integer index array order[i]=i and sort the indices using strcmp(names[order[j]], names[keyIndex]). Only integers are shifted. The names stay in their original storage. This reduces movement from up to L bytes per move to sizeof(int) bytes per move, at the cost of one index array and one extra indirection when comparing or displaying.

Names n	Binary-search comparisons upper bound	Sorting comparisons upper bound for insertion sort
100	7	4,950
500	9	124,750
5,000	13	12,497,500

The binary-search bounds are ceil(log2(n+1)). Insertion sort is used in the listing for clarity; for thousands of names, a faster O(n log n) sort such as qsort is normally preferable. Compare actual counters for copying names versus sorting indices; do not invent measured values. A 29-character name fits in NAME_SIZE=30 with the terminator; a 30-character name does not.

5. Student B – Design and Correctness

strcmp compares corresponding characters as unsigned-character values until a difference or a terminating null character is found. It returns a negative value, zero or a positive value; the exact magnitude is not important. In the usual execution character set, uppercase letters sort before lowercase letters, so "Apple" and "apple" are not equal and may appear in different positions.

A case-insensitive comparison can compare tolower((unsigned char)a[i]) and tolower((unsigned char)b[i]) until a difference or terminator. Apply the same comparison consistently during sorting, searching and duplicate detection. For non-English names, locale-aware Unicode collation is more complex than byte-wise strcmp.

Swapping two fixed-size name rows using strcpy copies characters into a temporary buffer and back. The cost is proportional to the lengths of the two names (bounded by NAME_SIZE). Swapping indices requires only a few integer assignments. After sorting, exact duplicates are adjacent because equal keys have the same sorted position; one scan detects each run.

6. Shared Group Work and Joint Conclusion

The group reads names safely, removes line endings, sorts them, performs binary search and scans adjacent rows for duplicates. Sorting indices or pointers saves time because a movement operation copies only an integer or address rather than up to 30 characters. The trade-off is extra index/pointer storage and indirection, and the original name array is not physically in alphabetical order; display must follow the sorted index list.

7. Alternative Concept – Pointer Array and qsort

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int compare_names(const void *p1, const void *p2) {
const char * const *a = p1;     const char * const
*b = p2;     return strcmp(*a, *b); }
/* Example: char storage[500][30]; char *list[500];
Fill storage safely, then set list[i] = storage[i].
qsort(list, count, sizeof list[0], compare_names); */
```

qsort typically performs O(n log n) comparisons; each strcmp may inspect up to L characters, so character-comparison work is O(n log n × L). A pointer swap is O(1), unlike copying full strings. The pointer list uses O(n) extra space (n pointers); qsort's own stack/workspace is implementation-dependent. The strings themselves remain in their original buffers.

8. Complexity Analysis

Operation / method	Time	Extra space
strcmp of two names	O(L)	O(1)
Binary search	O(L log n)	O(1)
Insertion sort of full names	O(n² × L) worst case	O(L) temporary key
qsort of pointers	O(n log n × L) typical	O(n) pointer list plus library workspace
Duplicate scan after sort	O(n × L)	O(1)

Q7 – Design an Election Counting Program from First Principles (10 Marks)

1. Stage 1 – Interpret the Problem (Joint)

Input: candidate count c (1 to 12), vote count v (0 to 5,000 in the stated system), and v integer candidate numbers. A vote is valid only when 1 ≤ vote ≤ c. Zero, negative values and values above c are invalid.

Output: valid vote count, invalid vote count, each candidate's vote total in descending order, winner, and whether the highest count is tied. Candidate numbers are used as identifiers. For equal vote totals, this implementation orders by smaller candidate number first.

Arrays: `count[12]` stores votes per candidate; `result[12]` stores candidate indices for sorting; the input stream is processed one vote at a time, so a vote array is not required. Requirements are measurable: exactly v entries are processed; valid + invalid = v; the sum of candidate counts = valid; every candidate appears once in the report; the reported winner has the maximum count; tie is true exactly when at least two candidates share the maximum.

2. Stage 2 – Design (Student A)

Use one pass over the votes. For each vote x, if x is within 1..c, increment count[x−1] and valid; otherwise increment invalid. Then sort candidate indices by descending count, using candidate number as a deterministic tie-break. The first candidate is a winner. A tie exists if another candidate has the same maximum count.

```
READ c, v initialise count[0..c-1] = 0 valid = 0; invalid = 0 repeat v times:
READ x     if 1 <= x <= c: count[x-1]++; valid++     else: invalid++ result =
[0, 1, ..., c-1] sort result by count descending; for equal counts, candidate
number ascending print valid, invalid and sorted candidate totals maximum =
count[result[0]] tie = any later result whose count equals maximum if maximum
== 0: report no votes / no elected winner else report all candidates with
maximum as winner(s)
```

3. Stage 3 – Complete C Program (Student B)

```
#include <stdio.h>

#define MAX_CANDIDATES 12
#define MAX_VOTES 5000

typedef struct {
int candidate;
int votes; }
Result;

void sort_results(Result r[], int c) {     /* Insertion sort: descending
votes, then ascending candidate number. */     for (int i = 1; i < c; i++) {
Result key = r[i];         int j = i - 1;         while (j >= 0 &&
(r[j].votes < key.votes ||               (r[j].votes == key.votes &&
r[j].candidate > key.candidate))) {             r[j + 1] = r[j];
j--;         }
        r[j + 1] = key;     } } int
main(void) {     int c, v, valid = 0,
invalid = 0;     int
count[MAX_CANDIDATES] = {0};
Result result[MAX_CANDIDATES];

    printf("Enter number of candidates (1-%d): ", MAX_CANDIDATES);
if (scanf("%d", &c) != 1 || c < 1 || c > MAX_CANDIDATES) {
printf("Invalid candidate count\n"); return 1;
    }     printf("Enter number of votes (0-%d): ",
MAX_VOTES);     if (scanf("%d", &v) != 1 || v < 0 || v >
MAX_VOTES) {         printf("Invalid vote count\n");
return 1;     }

    printf("Enter %d vote values:\n", v);     for (int
i = 0; i < v; i++) {         int x;         if
(scanf("%d", &x) != 1) {             printf("Invalid
input format\n"); return 1;         }         if (x >=
1 && x <= c) {             count[x - 1]++;
valid++;         } else {             invalid++;
}     }

    for (int i = 0; i < c; i++) {
result[i].candidate = i + 1;
```

```
result[i].votes = count[i];     }
sort_results(result, c);

    printf("Valid votes: %d\nInvalid votes: %d\n", valid, invalid);
printf("Candidate totals (descending):\n");     for (int i = 0; i <
c; i++)         printf("Candidate %d: %d\n",
               result[i].candidate, result[i].votes);

    if (valid == 0) {         printf("No valid votes; no winner can be
declared.\n");     } else {         int max_votes = result[0].votes;
int tied = 0;         for (int i = 1; i < c; i++)             if
(result[i].votes == max_votes) tied = 1;         if (tied) {
printf("Tie for first place:");             for (int i = 0; i < c &&
result[i].votes == max_votes; i++)                 printf(" Candidate
%d", result[i].candidate);             printf("\n");         } else {
printf("Winner: Candidate %d\n", result[0].candidate);         }     }
return 0; }
```

4. Sample Input and Output – Invalid Votes and Tie

```
Input:
3
8
1 2 3 1 2 0 4 -1

Output:
Valid votes: 5
Invalid votes: 3 Candidate
totals (descending):
Candidate 1: 2
Candidate 2: 2
Candidate 3: 1
Tie for first place: Candidate 1 Candidate 2
```

Hand check: votes 1 and 1 give Candidate 1 two votes; votes 2 and 2 give Candidate 2 two votes; vote 3 gives Candidate 3 one vote. The values 0, 4 and −1 are invalid. Thus valid=5, invalid=3 and the top count is tied.

5. Additional Test Cases

Case	Candidates	Votes	Expected result
No votes	3	none (v=0)	valid 0, invalid 0; no winner
All invalid	2	0, 3, -2	valid 0, invalid 3; no winner
Single candidate	1	1, 1, 0	candidate 1 has 2; invalid 1; winner 1
Case	**Candidates**	**Votes**	**Expected result**
Clear winner	3	1, 2, 1, 3, 1	candidate 1 has 3; winner 1
Tie	3	1, 2, 3, 1, 2	candidate 1 and 2 tie with 2

6. Stage 3 – Measurement Method

The assignment asks for timing 5,000 and 500,000 votes, although the stated system limit is 5,000. For the 500,000 experiment, temporarily raise the test harness limit and use a generated input stream; do not change the production requirement without approval. Measure only the counting/sorting section separately from terminal input/output, because console I/O can dominate the time. Use the same compiler and machine, repeat runs and report the median.

The count pass performs exactly v validity checks and at most v counter increments. Sorting at most 12 candidate records is negligible. If c is increased to 500, sorting remains small compared with large v. The code above uses insertion sort for the result records; it is $O(c^2)$, but $c \le 12$ in the required application.

Vote count v	Counting work	Candidate result sort (c=12)	Expected growth
5,000	5,000 vote inspections	At most 66 key comparisons in insertion	sortBaseline

500,000	500,000 vote inspections	At most 66 key comparisons in insertion	sortAbout 100× counting work

These are operation-count expectations, not measured seconds. The 500,000-vote case should take roughly proportional counting time if input generation and I/O are excluded. Record actual measured seconds from the lab machine in a separate results sheet.

7. Stage 4 – Defend the Design (Joint)

A count array is better than searching candidate names for every vote because the candidate number directly selects the count slot in O(1) time. Searching a candidate list would take up to O(c) comparisons per vote, or O(v·c) total. The count array is simple, fast and uses little memory.

For 500 candidates, enlarge the count and result arrays to 500, validate votes from 1 to 500, and choose a suitable O(c log c) sort if reporting order must be sorted. The remaining limitation is that candidate numbers must be valid, stable identifiers; the design also assumes each vote contains exactly one candidate number and does not address duplicate ballots, voter eligibility or audit/security controls.

8. Alternative Concept and Complexity Analysis

Search candidate names for every vote: for each vote, search the candidate list to find the matching candidate. This is O(v·c) time in the worst case and O(c) storage for candidate names/counts. A hash map could reduce expected lookup time but needs more complex collision handling.

Sort all votes and count runs: store all v votes, sort them, then scan equal-value runs. Sorting costs O(v log v), the scan costs O(v), and storage is O(v) for the vote array plus sort workspace. Invalid values must be filtered or reported separately before counting.

Count array: one pass takes O(v), then sorting candidate results takes O(c log c) with an efficient sort (or $O(c^2)$ with the simple insertion sort used here). Total is O(v+c log c) and extra space is O(c).

Approach	Time	Extra space	Comment
Count array	O(v + c log c)	O(c)	Best fit when IDs are compact inte
Search candidate list per vote	O(v·c)	O(c)	Simple but repeats candidate lookup
Sort all votes then count runs	O(v log v)	O(v)	Stores all votes; useful if sorted vote list is also needed

9. Overall Conclusion

The election program meets the stated counting rules: every input vote is classified exactly once, valid votes are accumulated by candidate number, totals are displayed in descending order, and the maximum is checked for ties. The design is efficient for up to 12 candidates and can be extended by increasing array capacity and choosing a scalable sort for larger candidate sets.